UNIVERSIDAD CASA GRANDE

MAESTRÍA EN INTELIGENCIA ARTIFICIAL Y CIENCIA DE DATOS

Paradigmas de Programación para Inteligencia Artificial y Análisis de Datos C2026 P1

TEMA:

Proyecto Final Integrador

Aplicación modular en Streamlit con agente de IA local y análisis de datos

Nombre:

Ronald Fabrizio Rodriguez Castro

Fecha de entrega:

15/10/2026

Maestría en Inteligencia Artificial y Ciencia de Datos

Índice de contenido

Resumen....2
Introducción....2
Objetivos....3
Marco teórico....3
Dataset y problema identificado....4
Metodología y arquitectura....5
Desarrollo de la aplicación....6
Integración del agente de IA....8
Resultados....9
Reproducibilidad y despliegue....10
Conclusiones....11
Referencias....11
Anexos....13
- Anexo A. Capturas de funcionamiento....13
- Anexo B. Repositorio de GitHub....14
- Anexo C. Video demostrativo....14

Índice de figuras

Figura 1. Arquitectura general de la aplicación....5
Figura 2. Interfaz principal de la aplicación....7
Figura 3. Tasa de impago según el grado del préstamo y la carga financiera....9
Figura A1. Riesgo de impago según el monto del préstamo en la aplicación....13
Figura A2. Respuesta del agente de IA a una pregunta sobre el dataset....13
Figura A3. Prompt enviado al modelo, mostrado como evidencia de la respuesta....14

Índice de tablas

Tabla 1. Resumen de calidad del dataset....5
Tabla 2. Reglas de limpieza aplicadas....7
Tabla 3. Resultados principales del análisis....10

Resumen

Este proyecto presenta una aplicación modular en Streamlit para explorar el riesgo de impago en préstamos personales, a partir del Credit Risk Dataset de Kaggle (32,581 solicitudes y 12 variables). La carga y limpieza de datos, el análisis exploratorio, las visualizaciones, el cliente de Ollama y el agente están en módulos independientes, unidos por una interfaz de cinco pestañas. Un agente de IA que funciona de forma local con Llama 3.2 responde preguntas en lenguaje natural: Python elige las herramientas y calcula las cifras, y el modelo solo las interpreta. En las pruebas, las cifras que citó fueron correctas, pero su interpretación tuvo errores que la evidencia permite detectar. Tras eliminar 165 duplicados y 5 edades imposibles, y completar casi 4,000 valores faltantes, quedaron 32,411 registros, de los cuales el 21.87 % terminó en impago. El riesgo crece con el grado del préstamo, del 9.96 % en el grado A al 70.54 % en el F, y con la carga financiera: quienes piden más del 30 % de su ingreso incumplen en el 70.44 % de los casos. El README y el archivo requirements.txt permiten reproducir el entorno en otro equipo.

Palabras clave: Streamlit, Ollama, agentes de IA, riesgo crediticio, análisis de datos, paradigmas de programación.

Introducción

Cada vez que una entidad financiera aprueba un préstamo, asume la posibilidad de que no se le pague. Saber qué solicitantes y qué tipos de préstamo tienen más probabilidad de incumplir ayuda a decidir mejor, y por eso el riesgo crediticio es un problema clásico del análisis de datos. Este proyecto busca responder: ¿qué características del solicitante y del préstamo se asocian con un mayor impago, y cómo se puede segmentar a los clientes según ese riesgo?

Para responderlo construí una aplicación en Streamlit que reúne la carga y limpieza de datos, el análisis, las visualizaciones y un agente de IA que corre en el propio equipo con Ollama. La lógica vive en módulos separados y el agente solo explica cifras que Python ya calculó. El dataset resultó apropiado porque mezcla variables numéricas y categóricas, tiene valores faltantes y atípicos reales, una variable objetivo binaria y un tamaño manejable en un computador personal.

Objetivos

Objetivo general

Desarrollar una aplicación modular en Streamlit para explorar y analizar un dataset de riesgo crediticio, integrando funciones reutilizables y un agente de IA local mediante Ollama que apoye la interpretación de los resultados.

Objetivos específicos

- Diseñar funciones para cargar, validar, limpiar, explorar y visualizar el dataset.
- Organizar la aplicación en módulos y clases que separen interfaz, datos, análisis y agente.
- Integrar un modelo local mediante Ollama que interprete resultados calculados por la aplicación.
- Documentar el entorno, las dependencias y la ejecución para que el proyecto sea reproducible.

Marco teórico

Streamlit y análisis de datos

Streamlit permite crear aplicaciones de datos con Python y sirve como capa de presentación de un flujo analítico; la lógica de carga, análisis y gráficos debe delegarse a

módulos especializados (Streamlit, s. f.). Los datos se manipulan con pandas (pandas development team, s. f.) y los gráficos se hicieron con Plotly (Plotly, s. f.), Seaborn y Matplotlib.

Ollama y agente local

Ollama ejecuta modelos de lenguaje en el propio equipo y los expone mediante una interfaz de programación. Se usó Llama 3.2 de 3 mil millones de parámetros, con la versión de 1B como alternativa para equipos limitados (Ollama, s. f.). El modelo no es un agente por sí solo: la lógica que elige herramientas, arma el contexto y muestra la evidencia se programó en Python.

Paradigmas de programación

El proyecto combina tres paradigmas: el imperativo, en las validaciones y ciclos; el funcional, en operaciones que reciben un DataFrame y devuelven un resultado sin modificar el original; y la orientación a objetos, en las clases que encapsulan el análisis, la comunicación con Ollama y el agente (Python Software Foundation, s. f.).

Dataset y problema identificado

Dataset seleccionado: Credit Risk Dataset (tahmidmir, s. f.). Fuente: https://www.kaggle.com/datasets/tahmidmir/credit-risk-dataset.

Justificación técnica: el riesgo crediticio es la posibilidad de que quien recibe un préstamo no lo pague, y aquí se aborda como segmentación descriptiva con *loan_status* (1 = impago, 0 = pagó) como objetivo. El dataset reúne siete variables numéricas y cuatro categóricas, lo que permite desde correlaciones hasta comparaciones entre segmentos. Su calidad es imperfecta de forma útil: hay nulos en dos variables, 165 duplicados y valores imposibles (una edad de 144 años, una antigüedad laboral de 123) e ingresos extremos. El mayor ingreso original, 6,000,000, pertenecía a un registro con edad imposible y se descartó; el

mayor ingreso válido, 2,039,784, se conserva. Solo el 21.8 % de los préstamos terminó en impago, un desbalance que exige leer las tasas con cuidado. La Tabla 1 resume el tipo de cada variable y sus nulos.

Tabla 1
Resumen de calidad del dataset

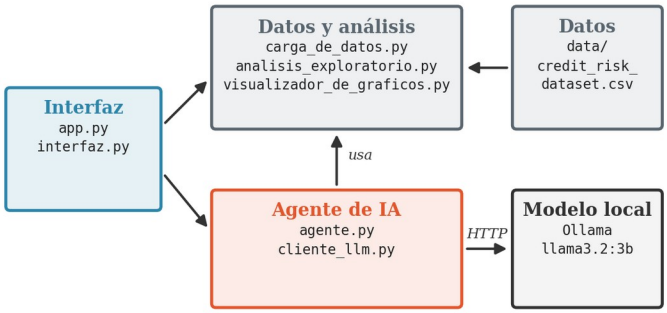
Variable	Tipo	Nulos	% nulos
person_age	Numérica	0	0.00 %
person_income	Numérica	0	0.00 %
person_home_ownership	Categórica	0	0.00 %
person_emp_length	Numérica	895	2.75 %
loan_intent	Categórica	0	0.00 %
loan_grade	Categórica	0	0.00 %
loan_amnt	Numérica	0	0.00 %
loan_int_rate	Numérica	3,116	9.56 %
loan_status	Binaria (objetivo)	0	0.00 %
loan_percent_income	Numérica	0	0.00 %
cb_person_default_on_file	Categórica	0	0.00 %
cb_person_cred_hist_length	Numérica	0	0.00 %

Nota. Elaboración propia con los datos originales (n = 32,581), antes de la limpieza.

Metodología y arquitectura

Cada módulo se escribió, se probó con los datos reales y se guardó en un commit antes de pasar al siguiente. La aplicación se organiza en capas que separan la interfaz, el tratamiento de datos y la integración con el modelo local (Figura 1), lo que permite probar cada parte por separado y evitar lógica repetida.

Figura 1
Arquitectura general de la aplicación



Nota. Elaboración propia. Las flechas indican qué módulo usa a cuál.

Estructura del repositorio

```
UCG_proyecto_final/
├── app.py
├── README.md
├── requirements.txt
├── .gitignore
├── .streamlit/config.toml
├── data/credit_risk_dataset.csv
├── assets/
└── src/
    ├── __init__.py
    ├── carga_de_datos.py
    ├── analisis_exploratorio.py
    ├── visualizador_de_graficos.py
    ├── cliente_llm.py
    ├── agente.py
    └── interfaz.py
```

app.py solo coordina y delega cada pestaña a *interfaz.py*, que se limita a dibujar. *carga_de_datos.py* lee y limpia el CSV, *analisis_exploratorio.py* concentra los cálculos en la clase AnalizadorRiesgo, *visualizador_de_graficos.py* produce las figuras, *cliente_llm.py* habla con Ollama y *agente.py* conecta las herramientas con el modelo. Una misma función alimenta a la pantalla y al agente: la tabla de impago por grado que se grafica es la que el agente recibe.

Apoyo de IA. Se utilizó un asistente de IA generativa (Anthropic, 2026) para plantear la estructura, revisar código y preparar borradores de redacción. La ejecución y prueba de cada módulo, la revisión de los resultados y las decisiones finales fueron del autor.

Desarrollo de la aplicación

Funciones analíticas

Cinco funciones concentran el análisis. *limpiar_datos* devuelve una copia limpia y una bitácora de lo que cambió (Tabla 2), sin modificar el original. *tasa_por_categoria* y *tasa_por_segmento* calculan clientes, impagos y porcentaje de impago por valor de una categórica o por tramos de una numérica. *correlacion_con_objetivo* mide la correlación de Pearson con el impago y *ranking_riesgo* ordena los segmentos por riesgo, descartando los de menos de 100 clientes.

Tabla 2
Reglas de limpieza aplicadas

Problema	Regla aplicada	Registros
Registros duplicados	Se eliminan	165
Edad mayor a 100 años	Se elimina el registro	5
Antigüedad laboral mayor a 60 años	Se marca como nulo y se imputa	2
Antigüedad laboral nula	Mediana global	889
Tasa de interés nula	Mediana del grado del préstamo	3,094

Nota. Elaboración propia. Los 889 registros con antigüedad nula incluyen los 2 marcados como inválidos. La tasa se imputa por grado porque ambas variables están muy ligadas.

Cada gráfico se eligió según el tipo de variable: dona para la binaria, barras con línea de promedio para comparar grupos, histograma con boxplot para las numéricas, y mapas de calor para correlaciones y cruces de categóricas.

Interfaz Streamlit

La interfaz tiene una barra lateral con el estado de Ollama y cinco pestañas: Resumen, Calidad de datos, Exploración, Agente de IA y Proyecto (Figura 2). Consume funciones de los módulos sin repetir cálculos y carga los datos una sola vez con el caché de Streamlit.

Figura 2
Interfaz principal de la aplicación



Nota. Captura de elaboración propia de la pestaña Resumen. Se ven los indicadores generales, la distribución de la variable objetivo y los hallazgos calculados a partir de los datos.

Integración del agente de IA

El agente trabaja siempre con información calculada, en cuatro pasos. Primero, *seleccionar_herramientas* busca palabras clave en la pregunta y elige hasta tres de las 13 herramientas autorizadas (riesgo por grado, correlaciones, calidad de datos, entre otras). Segundo, se ejecutan solo esas herramientas; cualquier otro nombre se ignora. Tercero, se arma un mensaje con los datos calculados y la pregunta. Cuarto, el cliente de Ollama lo envía con temperatura baja (0.2) para que las respuestas sean estables.

Al modelo nunca se le envía el dataset, solo cifras resumidas, con reglas claras: usar solo los números entregados, no inventar, comparar con la tasa global y hablar de asociación, no de causalidad. Decidí que Python eligiera las herramientas porque los modelos pequeños son poco fiables al devolver respuestas estructuradas; así el resultado es reproducible y basta una sola llamada.

Responsabilidad de la clase agente

```
class AgenteRiesgo:
    def responder(self, pregunta):
        nombres = self.seleccionar_herramientas(pregunta)
        resultados = self.ejecutar_herramientas(nombres)
        prompt = self.construir_prompt(pregunta, resultados)
        return self.cliente.generar(prompt, sistema=PROMPT_SISTEMA)
```

Es una versión simplificada; la real también captura errores de conexión. Para comprobar la coherencia, la aplicación muestra junto a cada respuesta su evidencia: las tablas calculadas, el gráfico y el prompt enviado, de modo que cada cifra puede contrastarse con Python. Si Ollama no está disponible, avisa y aun así muestra los datos.

Modelo local

Se usó llama3.2:3b (2.0 GB), descargado con *ollama pull llama3.2:3b*. La consulta de prueba de conexión tardó menos de un minuto y las dos consultas al agente tardaron 81.9 s y 78.6 s en el equipo de desarrollo. La barra lateral permite elegir otro modelo instalado, como llama3.2:1b, y avisa si falta el modelo o el servicio.

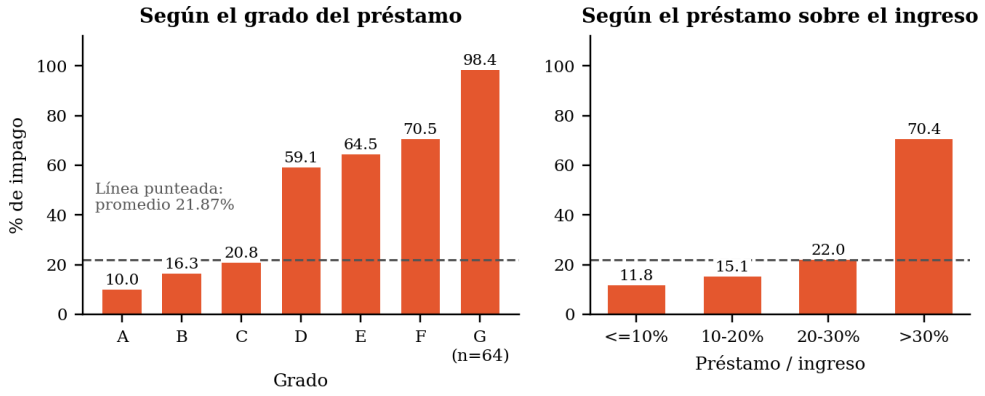
Resultados

Calidad de datos. Se descartaron 170 de 32,581 registros (0.5 %) y se completaron casi 4,000 valores faltantes; el análisis usa 32,411 clientes, de los cuales el 21.87 % terminó en impago.

Patrones de riesgo. El grado separa muy bien los casos: el impago va del 9.96 % en el grado A al 59.06 % en el D y al 70.54 % en el F. La carga financiera produce un salto parecido: quienes piden más del 30 % de su ingreso incumplen en el 70.44 % de los casos, frente al 11.76 % de quienes piden hasta el 10 % (Figura 3). También diferencian la vivienda (31.62 % si alquilan y 7.49 % si es propia) y el incumplimiento previo (37.87 % frente a 18.44 %), mientras que la edad casi no discrimina. Al cruzar variables, con grado D los préstamos para consolidar deudas y los médicos terminan en impago en el 92.8 % y el 87.9 % de los casos (598 y 655 clientes). Las correlaciones más altas con el impago son la carga financiera (0.38) y la tasa de interés (0.334); la del ingreso es negativa (−0.169).

Figura 3

Tasa de impago según el grado del préstamo y la carga financiera



Nota. Elaboración propia con los datos limpios (n = 32,411). El grado G tiene solo 64 clientes, por lo que su cifra es poco estable.

Agente de IA. Se preguntó: ¿Cómo influye el grado del préstamo en el impago? Con la herramienta *riesgo_loan_grade*, el agente respondió en 81.9 s que los grados A y B tienen las tasas más bajas (9.96 % y 16.32 %) y el F una de 70.54 %, sobre la global de 21.87 %. Las seis

cifras que citó coinciden con la tabla de Python (Figura A2). La interpretación, en cambio, falla: incluye el grado C (20.76 %) entre los de impago más alto aunque está bajo el promedio, omite los grados E (64.49 %) y G (98.44 %) y habla de un impacto «significativo» sin prueba estadística alguna.

Una segunda consulta libre, «dame el rango de edad que compra vivienda y su grado de impago», muestra un límite del enfoque. El dataset no registra quién compra vivienda, y la palabra «grado» activó la herramienta del grado del préstamo aunque se refería al nivel de impago. El modelo concluyó que el rango de 36 a 45 años compra vivienda por tener la menor tasa de impago (20.68 %), apenas 0.01 puntos bajo el de 26 a 35 años. Las cifras fueron correctas; la conclusión, no, y la evidencia visible permite detectarlo. La Tabla 3 resume los resultados principales.

Tabla 3
Resultados principales del análisis

Indicador	Resultado	Interpretación
Registros analizados	32,411	Se descartó el 0.5 % por duplicados o datos imposibles.
Variable con más nulos	loan_int_rate (9.56 %)	Se imputó con la mediana de su grado.
Tasa global de impago	21.87 %	Uno de cada cinco préstamos no se paga.
Relación destacada	r = 0.38 (préstamo / ingreso)	Es la mayor correlación; sobre el 30 % del ingreso el impago llega a 70.44 %.
Consulta al agente	Influencia del grado (81.9 s)	Las 6 cifras citadas coinciden con Python; la lectura omitió los grados E y G y ubicó mal el C.

Nota. Elaboración propia. Los resultados numéricos son de Python; la síntesis del agente fue contrastada con ellos.

Reproducibilidad y despliegue

Entorno local

El código está en el repositorio del Anexo B y su README.md permite reconstruir el proyecto: Python 3.10 o superior, entorno virtual, instalación desde requirements.txt, instalación de Ollama, descarga del modelo y ejecución. El dataset (1.8 MB) está en data/ y el

README indica cómo bajarlo de Kaggle; el modelo de Ollama no se guarda en el repositorio. Los comandos básicos son:

```
python -m venv .venv
.venv\Scripts\Activate.ps1      # en macOS o Linux: source .venv/bin/activate
pip install -r requirements.txt
ollama pull llama3.2:3b
streamlit run app.py
```

No se publicó en Streamlit Community Cloud, porque allí no existe el Ollama instalado en el computador del autor (Streamlit, s. f.); la versión del proyecto es local. Su funcionamiento se muestra en el video del Anexo C y en las capturas del Anexo A.

Conclusiones

La arquitectura modular cumplió su propósito: separar interfaz, datos, análisis, gráficos y agente permitió probar cada pieza por separado y reutilizar los mismos cálculos en la pantalla y en el agente.

El riesgo de impago se concentra en segmentos concretos: los grados D a F, los préstamos que superan el 30 % del ingreso, los solicitantes con incumplimiento previo y quienes alquilan; la edad aporta poco. Son asociaciones, no causas. El agente cumplió un papel de apoyo: al limitarlo a explicar cifras calculadas y mostrar su evidencia, sus respuestas pueden verificarse. En las pruebas todas las cifras citadas fueron correctas, pero la interpretación falló al ubicar mal un grado y al responder una pregunta que los datos no permiten contestar.

Como limitaciones, el agente elige herramientas por palabras clave (la palabra «grado» activó la herramienta equivocada en una consulta) y los grupos pequeños, como el grado G, dan cifras inestables. Como mejoras futuras: que el propio modelo elija las herramientas, agregar filtros globales a la interfaz y construir un modelo predictivo.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

Anthropic. (2026). *Claude* [Modelo de lenguaje de gran tamaño]. https://claude.ai

Ollama. (s. f.). *llama3.2*. https://ollama.com/library/llama3.2

pandas development team. (s. f.). *pandas documentation*. https://pandas.pydata.org/docs/

Plotly. (s. f.). *Plotly Python graphing library*. https://plotly.com/python/

Python Software Foundation. (s. f.). *Python documentation*. https://docs.python.org/3/

Streamlit. (s. f.). *Streamlit documentation*. https://docs.streamlit.io/

tahmidmir. (s. f.). *Credit risk dataset* [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/tahmidmir/credit-risk-dataset

Anexos

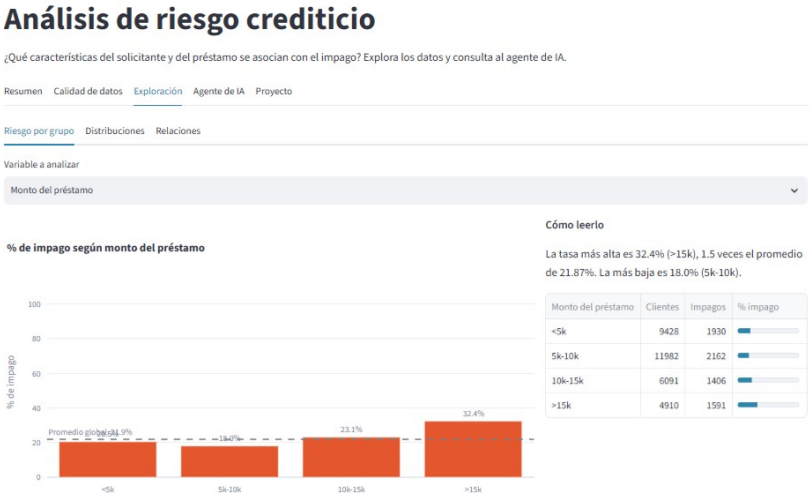
Los anexos reúnen la evidencia de funcionamiento y reproducibilidad del proyecto, con las figuras presentadas según el formato APA 7 (American Psychological Association, 2020).

Anexo A. Capturas de funcionamiento

Las capturas muestran la exploración de los datos y la interacción con el agente. La interfaz principal está en la Figura 2.

Figura A1

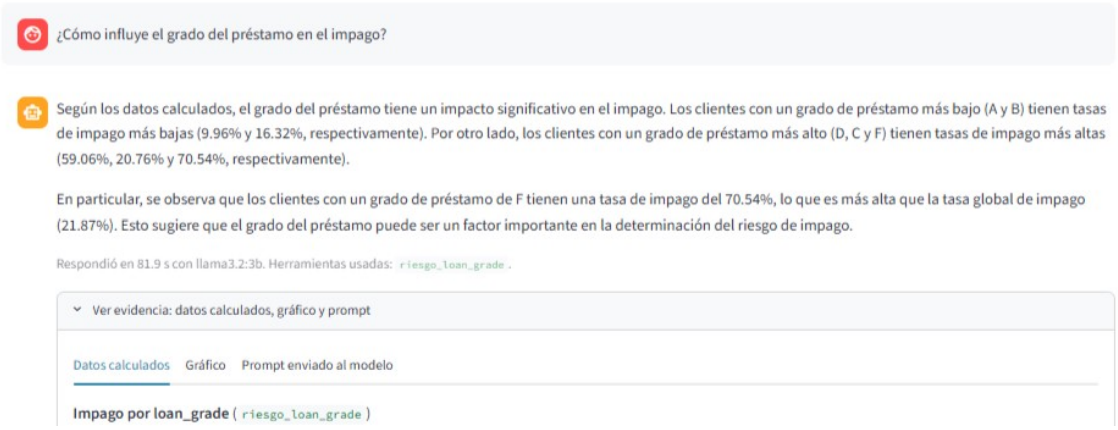
Riesgo de impago según el monto del préstamo en la aplicación



Nota. Captura de elaboración propia de la pestaña Exploración. Las barras muestran el porcentaje de impago por tramo de monto y la línea punteada, el promedio global; a la derecha se leen los valores calculados.

Figura A2

Respuesta del agente de IA a una pregunta sobre el dataset



Nota. Captura de elaboración propia de la pestaña Agente de IA. Se ven la pregunta, la respuesta de llama3.2:3b, el tiempo de respuesta (81.9 s) y la herramienta usada.

Figura A3

Prompt enviado al modelo, mostrado como evidencia de la respuesta



Nota. Captura de elaboración propia del desplegable de evidencia: instrucciones del sistema y bloque de datos calculados que recibió el modelo.

Anexo B. Repositorio de GitHub

Repositorio del proyecto: https://github.com/ucg-rfrodriguezc/UCG_proyecto_final

Contiene app.py, la carpeta src/ con los módulos, requirements.txt, README.md, .gitignore y el dataset.

Anexo C. Video demostrativo

Video de demostración (máximo 5 minutos), alojado en Google Drive: https://drive.google.com/file/d/19-RqUtLmYa16o6PIX5KyII_PZxUpP-he/view?usp=drive_link

El video muestra el inicio de la aplicación, la carga del dataset, el análisis y las visualizaciones, las funciones relevantes, la interacción con el agente, la conexión con Ollama y la interpretación de una respuesta.